



INSTITUTO
UNIVERSITÁRIO
DE LISBOA

Planeamento, arquitetura e implementação de um processo de ETL para um sistema de Data Warehouse/Business Intelligence

Mestrado em Sistemas Integrados de Apoio à Decisão

Unidade Curricular de Sistemas de Informação Analíticos II

Autores:

Eduardo Parracho, 111423

Guilherme Bravo, 100635

Luiz Vieira, 104570

Ruben Cabeça, 140183

Junho, 2026

Índice

1. Introdução e Apresentação do Caso de Estudo	2
1.1. <i>Descrição da Organização</i>	2
1.2. <i>Processo de Negócio Escolhido</i>	2
1.3. <i>Alterações ao trabalho de SIA I</i>	2
1.3.1 Modelo Relacional	2
1.3.2 Modelo Dimensional	3
2. Modelo Dimensional Elementar	3
2.1. <i>Modelo Dimensional Elementar Selecionado</i>	3
3. Fontes de Informação e Geração de Dados	5
3.1. <i>Fontes de Informação</i>	5
3.2. <i>Criação de Dados Operacionais</i>	5
3.3. <i>Análise de Perfil dos Dados (Data Profiling)</i>	6
4. Esquema Agregado	6
4.1. <i>Justificação da Implementação</i>	6
4.2. <i>Modelo Agregado</i>	6
4.3. <i>Matriz de Métricas Vs. Dimensões</i>	6
4.4. <i>Mais-valias da Implementação</i>	7
5. Sistema de ETL	7
5.1 <i>ETL Inicial e Incremental</i>	8
5.2 <i>Descrição detalhada de cada povoamento de dimensão</i>	10
5.2.1 DateDimension	10
5.2.2 DimHora	10
5.2.3 DimTipodeMovimento	10
5.2.4 DimTrabalhador	10
5.2.5 DimProduto	11
5.2.6 DimEntidade	11
5.3. <i>Nível de Processo</i>	12
5.4. <i>Nível de Partner</i>	12
5.5. <i>Carregamento da Tabela de Factos</i>	13

1. Introdução e Apresentação do Caso de Estudo

O presente relatório documenta o trabalho realizado no âmbito da Unidade Curricular de Sistemas de Informação Analíticos II (SIA II). O objetivo central consiste no planeamento, arquitetura e implementação de um processo de ETL (Extract, Transform, Load) para um sistema de Data Warehouse/Business Intelligence, tendo como base o trabalho de modelação dimensional desenvolvido em SIA I.

1.1. Descrição da Organização

A MotorData Car Dealership S.A. (*LionSupply* Concessionários), fundada em 2014 e sediada em Lisboa, é uma rede nacional de concessionários Peugeot com mais de 1000 trabalhadores distribuídos por seis regiões (Lisboa, Porto, Aveiro, Faro, Coimbra, Braga). A sua atividade abrange, venda de veículos novos e usados, serviços de manutenção e reparação, gestão e comercialização de peças e acessórios, garantia pós-venda, e partilha de veículos elétricos. A estrutura organizacional inclui áreas funcionais de Gestão de Vendas, Serviços e Reparações, Stock de Peças, Test Drives, Automóveis Elétricos e Recursos Humanos.

1.2. Processo de Negócio Escolhido

O processo de negócio selecionado, mantido relativamente a SIA I, é a Gestão de Stock de Peças, compreendendo os subprocessos de Entrada e Saída de Produtos. A gestão eficaz do stock é crítica: o excesso gera capital parado, enquanto ruturas causam perda de vendas e atrasos em reparações. O Data Mart a desenvolver visa capacitar os decisores a analisar movimentações de stock, monitorizar o desempenho operacional por concessionário e a nível de rede, e avaliar a eficiência e eficácia das operações.

1.3. Alterações ao trabalho de SIA I

Na sequência da revisão do trabalho desenvolvido em SIA I, foram implementadas alterações aos modelos dados operacional e dimensional.

1.3.1 Modelo Relacional

1. Hierarquia de produto na fonte

O modelo inicial não contemplava tabelas de categoria e subcategoria na fonte relacional. Foram agora introduzidas as entidades categoria, subcategoria e marca, estabelecendo a hierarquia produto, subcategoria, categoria. Esta alteração garante consistência estrutural entre o modelo relacional e o dimensional, além de suportar corretamente os processos de lookup no ETL.

2. Introdução da entidade armazém

A entidade armazém foi adicionada ao modelo, incluindo atributos como identificação, nome, concessionário associado, área total e responsável. Esta entidade permite representar explicitamente a infraestrutura física de armazenamento e suporta análises relacionadas com ocupação e valor de stock.

3. Introdução da entidade oficina

Foi criada a entidade oficina, com atributos semelhantes aos do armazém e associação ao concessionário. A sua inclusão permite modelar corretamente os fluxos de saída relacionados com reparações, suportando análises operacionais específicas.

4. Gestão de cargos do trabalhador

O modelo evoluiu de uma estrutura simples, baseada em id_cargo e função, para uma abordagem mais completa com a introdução de atributos adicionais e da tabela normalizada vinculo_profissional, que regista o historial de cargos ao longo do tempo. Esta evolução enriquece a DimTrabalhador com informação mais detalhada e representativa da realidade organizacional.

1.3.2 Modelo Dimensional

O modelo dimensional selecionado para implementação do ETL é a *FactMovimentoStock*, tabela de factos transacional que regista cada movimento individual de stock (entrada, saída), realizado num determinado concessionário ou armazém, numa data e hora concretas.

1. Reestruturação da FactMovimentoStock

As dimensões de concessionário e fornecedor foram fundidas numa única dimensão, DimEntidade. Esta abordagem reduz redundância e permite análises integradas entre diferentes tipos de entidades.

2. Reclassificação do custo unitário

O custo unitário foi removido da tabela de factos por se tratar de um atributo descritivo do produto e não de uma métrica transacional. A sua inclusão na tabela de factos seria incorreta do ponto de vista da modelação dimensional, sendo o seu lugar natural na DimProduto. O CustoTotal, calculado a partir da quantidade movimentada e do preço unitário, constitui a métrica adequada a registar na tabela de factos.

3. Simplificação da DimTipoMovimento

A dimensão foi reduzida aos atributos essenciais: tipo (entrada ou saída) e motivo de saída. Foram eliminados atributos inadequados, como quantidade, que pertence à tabela de factos.

2. Modelo Dimensional Elementar

2.1. Modelo Dimensional Elementar Selecionado

Modelo selecionado: FactMovimentoStock (Tabela de Factos Transacional), conforme ilustrado na Figura 1 [Link dbdiagram.io](http://dbdiagram.io).

Grão: Uma linha por linha de documento.

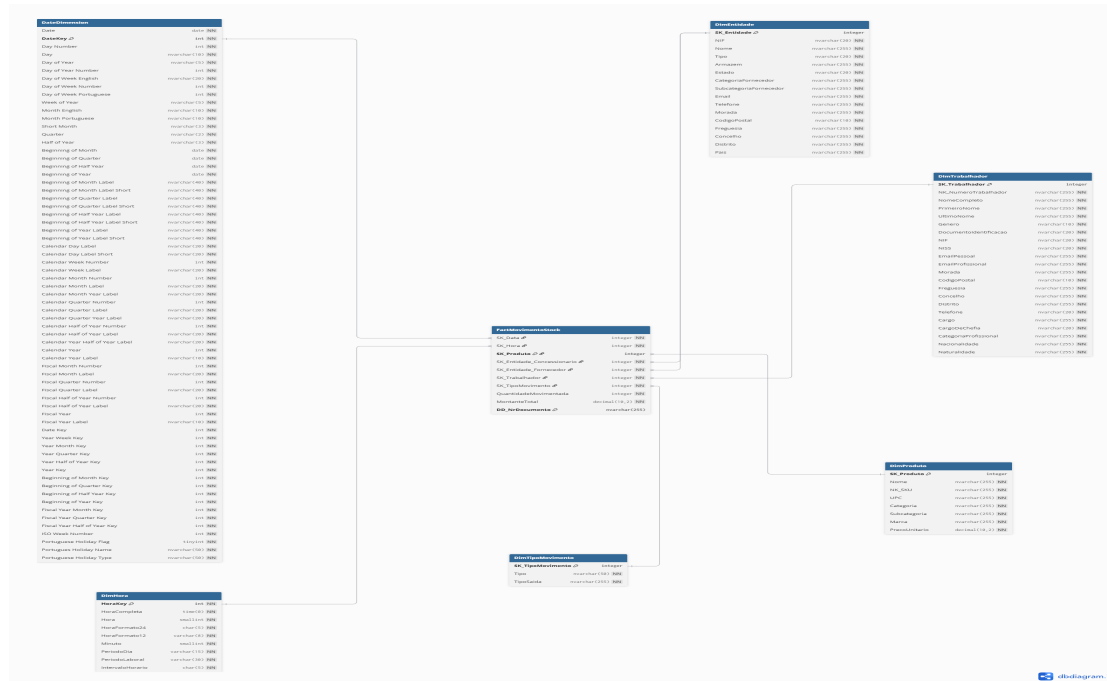


Figura 1 - Modelo dimensional de movimentação de stock.

O modelo é composto por uma tabela de factos, que inclui uma dimensão degenerada (DD_NrDocumento), e pelas seguintes dimensões:

- **DimData:** Dimensão conformada, com granularidade ao nível do dia.
- **DimHora:** Dimensão conformada, com granularidade ao nível do minuto.
- **DimProduto:** Catálogo de produtos, incluindo atributos como SKU, UPC, categoria, subcategoria e marca.
- **DimTrabalhador:** Registo dos trabalhadores responsáveis pelas operações de armazém, contendo informação pessoal, profissional e indicadores de função de chefia.
- **DimTipoMovimento:** Dimensão que fornece o contexto semântico do movimento, detalhando o tipo de operação e, quando aplicável, o tipo de saída.
- **DimEntidade (Dimensão Role-Playing):** Dimensão unificada que agrega informação geográfica e organizacional. Esta dimensão é utilizada em modo role-playing na tabela de factos, através de duas ligações distintas, assumindo os seguintes papéis:
 - **Concessionário (SK_Entidade_Concessionario),** para identificar a entidade da rede LionSupply associada aos movimentos de saída de produtos;
 - **Fornecedor (SK_Entidade_Fornecedor),** para identificar a entidade fornecedora nos movimentos de entrada.

Métricas (Factos): A tabela de factos armazena as medidas atómicas de cada transação de inventário, nomeadamente QuantidadeMovimentada e CustoTotal.

3. Fontes de Informação e Geração de Dados

3.1. Fontes de Informação

O sistema de *Data Warehouse* recorre a três fontes de informação heterogêneas. A fonte principal é a base de dados relacional operacional ([Figura 2](#)), implementada em SQL Server, que alberga todas as tabelas sistema fonte de gestão de stock: produtos, trabalhadores, fornecedores, armazéns, oficinas, concessionários e os movimentos de entrada e saída. A segunda fonte é um ficheiro Excel utilizado para popular a dimensão DimHora, contendo os atributos temporais ao nível da hora (e.g., hora, minuto, período do dia) que não se encontram presentes na base de dados operacional. A terceira fonte é um ficheiro Excel que serve de base ao carregamento da dimensão DimTipoMovimento, contendo os tipos de movimento de stock possíveis no sistema, nomeadamente a distinção entre movimentos de entrada e de saída de peças nos armazéns da rede.

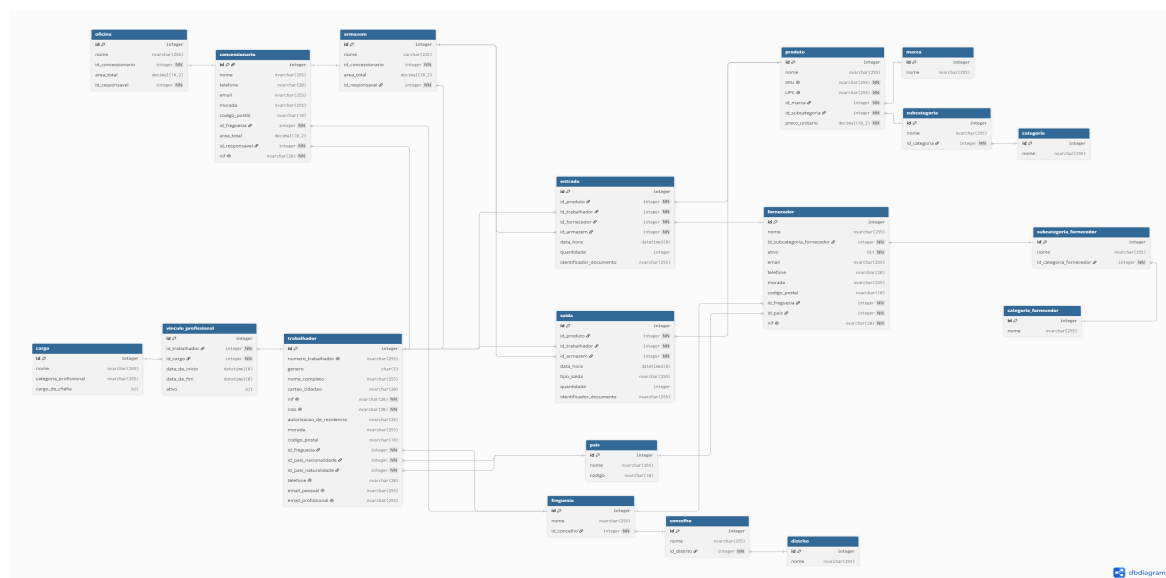


Figura 2 – Modelo de dados Relacional

3.2. Criação de Dados Operacionais

A criação dos dados operacionais foi realizada com recurso a Inteligência Artificial Generativa (*GenAI*). O processo assentou num conjunto iterativo de *prompts* que orientaram o modelo na geração das tabelas da base de dados, bem como no seu preenchimento com dados representativos do cenário de negócio. Apesar de não apresentarmos nenhum *prompt*, o script final gerado e utilizado para povoar a base de dados operacional foi partilhada como um dos artefactos entregues, e possui o nome *Grupo 3 script generate tables and data.sql*.

Com o objetivo de aumentar, ligeiramente, a complexidade do conjunto de dados, foi introduzida deliberadamente uma anomalia controlada: na coluna “gênero” da tabela “trabalhador”, as últimas 50 linhas foram definidas como valores *NULL*. Esta abordagem permite simular um problema real de qualidade de dados, o qual deverá ser identificado, tratado e devidamente documentado durante o processo de ETL, nomeadamente através da aplicação de uma regra de substituição por um valor “desconhecido”.

3.3. Análise de Perfil dos Dados (Data Profiling)

A análise de perfil de dados foi conduzida sobre o esquema *dbo* (SQL Server), recorrendo a um *script* SQL. O *script* mencionado percorre sistematicamente todas as tabelas e colunas, calculando, por coluna, o total de registos, a contagem e percentagem de valores nulos, a cardinalidade, os valores mínimo e máximo, bem como os comprimentos mínimo e máximo.

Os resultados completos da análise de perfil encontram-se exportados no ficheiro *Grupo_3_Data_Profiling.xlsx*, disponível para consulta na pasta de artefactos submetida com o presente trabalho.

De forma resumida, a qualidade dos dados é maioritariamente boa, uma vez que estes são dados fictícios, gerados por inteligência artificial. Como mencionado anteriormente, introduzimos alguns valores nulos só para ficarmos mais próximos de um caso real.

O *script* SQL completo encontra-se no Grupo_3_Script_Data_Profiling, disponível na pasta de artefactos.

4. Esquema Agregado

4.1. Justificação da Implementação

O esquema agregado foi concebido com o objetivo de melhorar o desempenho analítico da solução proposta, suportando sobretudo a camada tática e estratégica de análise.

4.2. Modelo Agregado

Foi criada a tabela de factos agregada *dw.FactMovimentoStockMensalAgg*, com o seguinte grão:

- Uma linha por Produto × Entidade × Mês × Tipo de Movimento.

As dimensões incluídas são:

- DimData (Ano, Semestre, Trimestre e Mês);
- DimProduto;
- DimEntidade;
- DimTipoMovimento.

As dimensões DimHora e DimTrabalhador não foram consideradas para o nível de agregação por não contribuírem para as análises identificadas na matriz de métricas.

4.3. Matriz de Métricas Vs. Dimensões

A decisão de implementar um esquema agregado baseou-se na análise da matriz Métricas vs. Dimensões apresentada na *Figura 3*. Esta matriz permitiu identificar que a métrica central para a tomada de decisão é o total de quantidade movimentada, analisável pelas dimensões de tempo ao nível do mês, produto, entidade concessionária e tipo de movimento, justificando a criação de um agregado dedicado a este nível de granularidade.

Matriz de Métricas vs. Dimensões — MotorData Car Dealership (SIA II)								
Métrica / Dimensão	Tipo	Data	Hora	Produto	Entidade (Conc. + Forn.)	Trabalhador	Tipo de Movimento	Incluir?
A — Métricas Elementares								
Quantidade de Entrada	#	X	X	X	X	X	X	S
Quantidade de Saída	#	X	X	X	X	X	X	S
Valor de Entrada (€)	€	X		X	X			S
Valor de Saída (€)	€	X		X	X		X	S
B — Métricas Agregadas								
Quantidade Total movimentada	SUM	X		X	X		X	S

Figura 3 – Matriz de Métricas vs. Dimensões

4.4. Mais-valias da Implementação

A implementação do esquema agregado proporciona diversas vantagens:

1. Melhoria de desempenho

A redução da granularidade dos dados permite executar consultas analíticas sobre um volume significativamente inferior de registos, reduzindo os tempos de resposta.

2. Simplificação do modelo analítico

A eliminação de dimensões pouco relevantes para a análise estratégica reduz a complexidade do modelo e facilita a construção de *dashboards* e relatórios.

3. Separação entre utilização operacional e analítica

Os utilizadores operacionais continuam a recorrer ao modelo transacional para análise detalhada dos movimentos, enquanto os utilizadores táticos e estratégicos utilizam preferencialmente o esquema agregado.

4. Maior facilidade de exploração em ferramentas BI

O menor número de dimensões e o nível de agregação mensal simplificam a criação de indicadores, visualizações e análises multidimensionais.

5. Sistema de ETL

O mapeamento detalhado entre colunas fonte e colunas de destino, incluindo regras de transformação, tipos SCD e regras de qualidade de dados por atributo, encontra-se documentado no ficheiro *Grupo_3_SourceToTarget_Final.xlsm*, submetido na pasta de artefactos.

As estruturas de dados de suporte ao pipeline ETL incluem a tabela *dsa.Control_ETL_Table*, que regista a marca temporal da última execução bem-sucedida por tabela fonte, a tabela *dsa.cdc_states*, que guarda o estado de ativação do CDC por tabela rastreada, nomeadamente *dbo.Trabalhador*, *dbo.Produto*, *dbo.Concessionario* e *dbo.fornecedor*, as tabelas de staging no

esquema DSA (uma por dimensão e por tabela de factos), e as tabelas de mapeamento de *Surrogate Keys*, que associam de forma persistente cada *Business Key* à respetiva SK gerada.

5.1 ETL Inicial e Incremental

O processo de ETL é dividido em inicial e incremental.

O ETL inicial, ilustrado na *Figura 4*, é composto por uma sequência de *containers* ligados por restrições de precedência de sucesso, garantindo que cada *container* apenas é executado após a conclusão bem-sucedida do anterior. A *task* de limpeza da tabela de controlo ETL corre em paralelo com o primeiro contentor, uma vez que não depende dele, e deve estar vazia neste momento para que o processo de carga inicial extraia a totalidade dos dados das fontes.

O primeiro *container* é responsável pela limpeza da tabela de factos. A razão pela qual a tabela de factos é limpa antes das dimensões prende-se com a integridade referencial: a tabela de factos contém chaves estrangeiras que apontam para as chaves primárias das dimensões, pelo que eliminar as dimensões em primeiro lugar resultaria numa violação dessas restrições.

O segundo *container* é responsável pela limpeza das tabelas de dimensão. Para cada dimensão, é executado um comando SQL de eliminação independente, correndo todas as *tasks* em paralelo. Assim que cada *task* de limpeza conclui com sucesso, é executado um segundo comando SQL que insere a linha especial de membro desconhecido na dimensão correspondente, seguindo a boa prática de *Kimball* de garantir que existe sempre um registo para os casos em que não é possível estabelecer uma associação.

O terceiro *container* povoa as dimensões. As dimensões estáticas `DimData`, `DimHora` e `DimTipoMovimento` são carregadas diretamente através de *dataflows*, sem recurso a tabelas de staging, uma vez que os seus valores são fixos e predefinidos. As restantes dimensões são carregadas a partir da DSA: os dados são primeiro copiados da fonte para a tabela de staging correspondente e, após um conjunto de transformações num *dataflow*, carregados na dimensão de destino. Antes de cada carregamento, as tabelas DSA e a tabela de mapeamento de Surrogate Keys são limpas, evitando duplicados na DSA e violações de chave primária no mapeamento. No final deste contentor, o estado inicial do CDC é guardado na tabela `dsa.cdc\_states`, assegurando que a primeira execução do ETL incremental parte do estado resultante desta carga completa.

Por fim, o quarto *container* carrega a tabela de factos. A DSA da tabela de factos é igualmente limpa antes do carregamento, pelo mesmo motivo aplicado às dimensões.

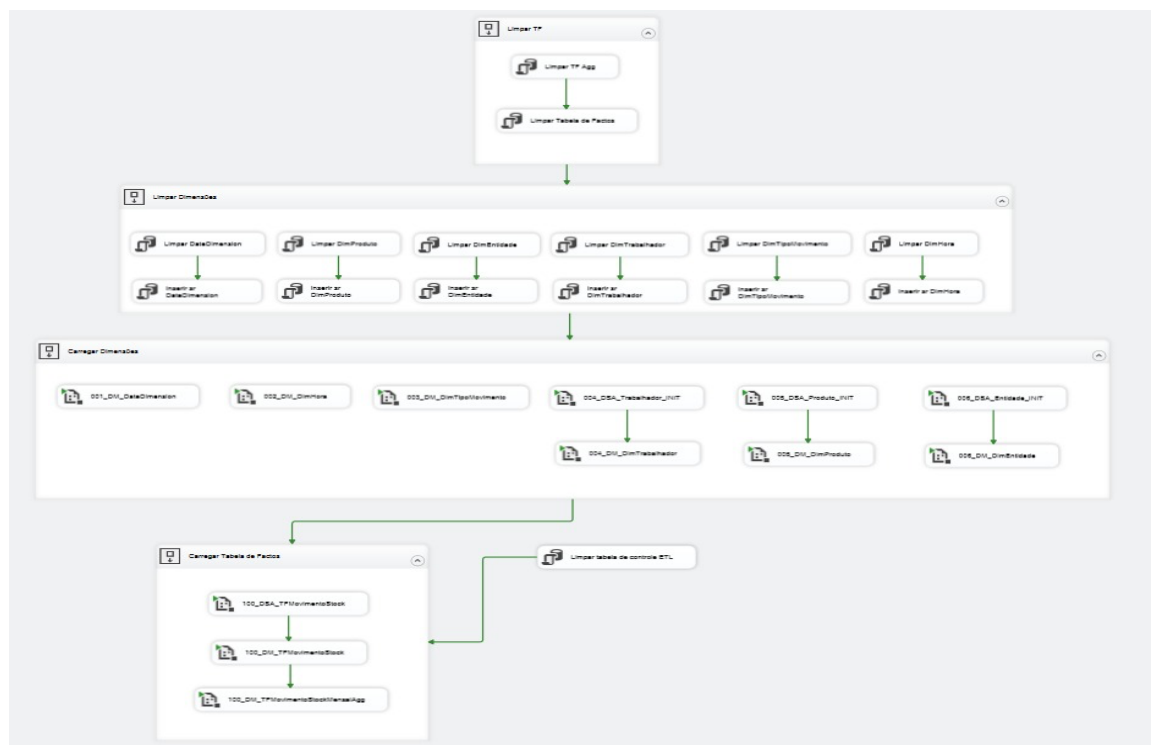


Figura 4 – ETL Inicial

O ETL incremental, ilustrado na *Figura 5*, segue a mesma lógica do ETL inicial, mas sem realizar a limpeza prévia das tabelas. O primeiro *container* atualiza em paralelo as dimensões sujeitas a histórico, nomeadamente DimTrabalhador, DimProduto e DimEntidade, extraíndo apenas os registos alterados desde a última execução com base na marca temporal da *Control_ETL_Table*. As dimensões estáticas não são recarregadas. Após a conclusão das dimensões, o segundo contentor carrega os registos novos na tabela de factos.



Figura 5 – ETL Incremental

5.2 Descrição detalhada de cada povoamento de dimensão

5.2.1 DateDimension

A DimData é uma dimensão estática cujo conteúdo não depende de dados operacionais. O seu povoamento é realizado através de um ciclo iterativo que percorre um intervalo de anos pré-definido, gerando um registo por dia até que todo o período temporal esteja representado na dimensão.

5.2.2 DimHora

Esta dimensão é estática e povoada com recurso a um ficheiro Excel que contém a dimensão na sua totalidade, com granularidade a nível do minuto. Este pacote é constituído apenas por um *Data Flow*, que pega nos dados da fonte (Excel), aplica uma conversão de dados (mudança de *data type* para *string* DT_STR para as colunas que possuem palavras com acento) e por fim o carregamento para a dimensão no esquema dw.

5.2.3 DimTipoDeMovimento

A dimensão Tipo Movimento segue a mesma lógica da DimHora.

5.2.4 DimTrabalhador

O pacote SSIS responsável pelo carregamento da DimTrabalhador é composto por um único *data flow*. A fonte de dados é a tabela *dsa.Trabalhador*, que replica a estrutura da tabela operacional. Através de um comando SQL que faz o tratamento de dados como transformar o género em texto descritivo, obter uma coluna com o primeiro e o último nome e o preenchimento da coluna documento de identificação com o primeiro valor não nulo entre cartão de cidadão e título de residência, considerando que apenas um destes atributos poderá estar em falta.

O fluxo inicia-se com um *lookup* à tabela de mapeamento *dsa.Trabalhador_sk*, com o objetivo de associar a cada registo a respetiva Surrogate Key previamente atribuída. De seguida, os valores nulos no atributo género são substituídos por 'n/a' através de uma transformação de dados.

Uma vez que a DSA preserva os identificadores numéricos da fonte, são realizados lookups às tabelas de referência correspondentes, permitindo converter cada identificador no respetivo valor descritivo.

O atributo CargoDeChefia, originalmente do tipo binário, é transformado em texto através de uma coluna derivada com lógica condicional, assumindo os valores 'Chefe' ou 'Comum'.

Após a conclusão das transformações, é aplicada a componente SCD de acordo com as regras definidas no source-to-target mapping. Para os atributos sujeitos a historização, são executados comandos SQL adicionais que removem as Surrogate Keys desatualizadas, geram novas entradas na tabela de mapeamento e substituem as chaves anteriores pelas novas, garantindo a consistência do histórico de versões na dimensão.

Por fim, os dados transformados são carregados na tabela DM.DimTrabalhador.

5.2.5 DimProduto

O pacote SSIS responsável pelo carregamento da DimProduto segue a mesma lógica descrita para a DimTrabalhador. Neste caso, como os dados não apresentam problemas de qualidade, não são necessárias transformações para tratamento de valores nulos nem conversões de tipo.

O fluxo consiste essencialmente na execução de *lookups* às tabelas de referência, com o objetivo de substituir os identificadores numéricos provenientes da DSA pelos respetivos valores descritivos, nomeadamente categoria, subcategoria e marca.

A componente SCD é aplicada de acordo com a mesma abordagem, incluindo a execução de comandos SQL para gestão de *Surrogate Keys*, assegurando a consistência do histórico de versões na dimensão. Finalmente, os dados já transformados são carregados na tabela DM.DimProduto.

5.2.6 DimEntidade

O pacote SSIS responsável pelo carregamento da DimEntidade segue uma abordagem semelhante à da DimTrabalhador, com a particularidade de a fonte de dados ser obtida através de um comando SQL que combina as tabelas *dbo.concessionario* e *dbo.fornecedor* por meio de um *UNION ALL*.

Esta consulta inclui já transformações iniciais em SQL, nomeadamente a criação de uma coluna textual que identifica o tipo de entidade, distinguindo concessionários de fornecedores.

O fluxo prossegue com a execução de *lookups* às tabelas de referência, com o objetivo de substituir os identificadores numéricos pelos respetivos valores descritivos. Dado que o *UNION ALL* agrega entidades de natureza distinta, alguns atributos são aplicáveis apenas a um dos tipos, originando valores nulos no outro. Estes valores são tratados através da sua substituição por 'n/a'.

Adicionalmente, é aplicada uma transformação de conversão de dados nos atributos que contêm caracteres acentuados, garantindo a compatibilidade de codificação com o esquema de destino.

Por fim, a componente SCD é aplicada de acordo com a mesma lógica das restantes dimensões, incluindo a gestão de *Surrogate Keys* para assegurar a consistência do histórico de versões, sendo os dados posteriormente carregados na tabela DM.DimEntidade.

5.3. Nível de Processo

A *Figura 6* apresenta o fluxo ETL ao nível do processo, evidenciando o pipeline de carregamento do Data Mart. O processo inicia-se com a execução em paralelo das seis dimensões, DimData, DimHora, DimEntidade, DimTrabalhador, DimProduto e DimTipoMovimento, sendo este paralelismo possível porque não existem dependências entre elas. Cada dimensão está num *container* independente, e o carregamento da tabela de factos *FactMovimentoStock* só é desencadeado após a conclusão bem-sucedida de todos os *containers* das dimensões, através de restrições de precedência de sucesso. Esta dependência sequencial é tecnicamente necessária, uma vez que o carregamento da tabela de factos requer a resolução das *Surrogate Keys* de todas as dimensões associadas, efetuada através de uma pesquisa de SK antes da inserção de cada registo de facto.

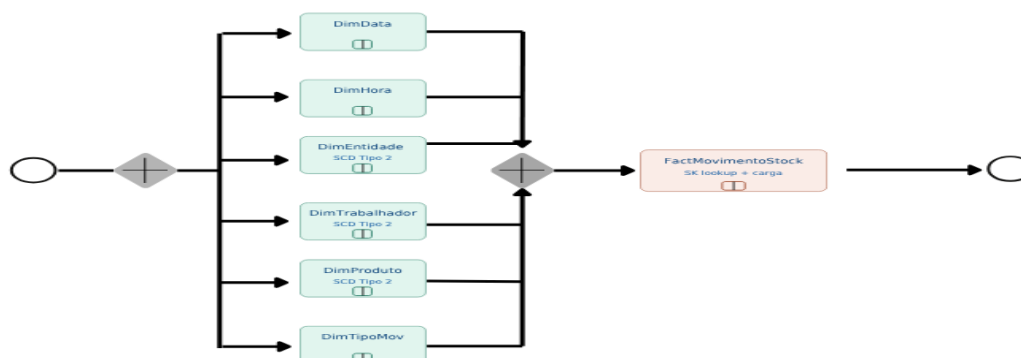


Figura 6 – Diagrama BPMN – Nível de Processo

5.4. Nível de Partner

A *Figura 7* descreve o padrão de carregamento aplicado a todas as dimensões não estáticas, ilustrado com o exemplo da DimTrabalhador. O fluxo é composto por três sub-processos sequenciais: CDC, SKG e SCD.

Na etapa de CDC, são identificadas as alterações ocorridas nas tabelas de origem desde a última execução ETL, com o estado de ativação por tabela registado em *dsa.cdc_states*. Na carga inicial, todos os registos são extraídos na íntegra; nas execuções subsequentes, apenas são processados os registos inseridos, modificados ou eliminados. Os dados extraídos são materializados na DSA, onde ficam disponíveis para as etapas seguintes. A atribuição de Surrogate Keys é então realizada com recurso a uma tabela de mapeamento persistente na DSA, que associa cada Business Key a uma SK gerada por auto-incremento. *BKs* já conhecidas mantêm o identificador existente, garantindo a estabilidade dimensional ao longo do tempo.

Antes do carregamento, os dados da DSA são sujeitos a um conjunto de transformações que incluem *lookups* para resolução de chaves, conversão de tipos, tratamento de nulos, substituição de identificadores numéricos por valores descritivos, conversão de campos binários em representações legíveis e criação de colunas derivadas. O carregamento segue depois a estratégia *SCD Type 2* para atributos que pretendemos guardar o histórico, gerindo versões através de *Start_Date* e *End_Date*, e *SCD Type 1* para atributos cuja alteração não justifica preservação de histórico, aplicando uma atualização direta ao registo existente.

DimData e DimHora são pré-populadas e não seguem este padrão. DimTipoMovimento é carregada a partir de um ficheiro Excel e, dado o seu carácter estático e a cardinalidade reduzida de três registos, procede-se à sua carga integral em cada execução, sem aplicação de extração incremental.

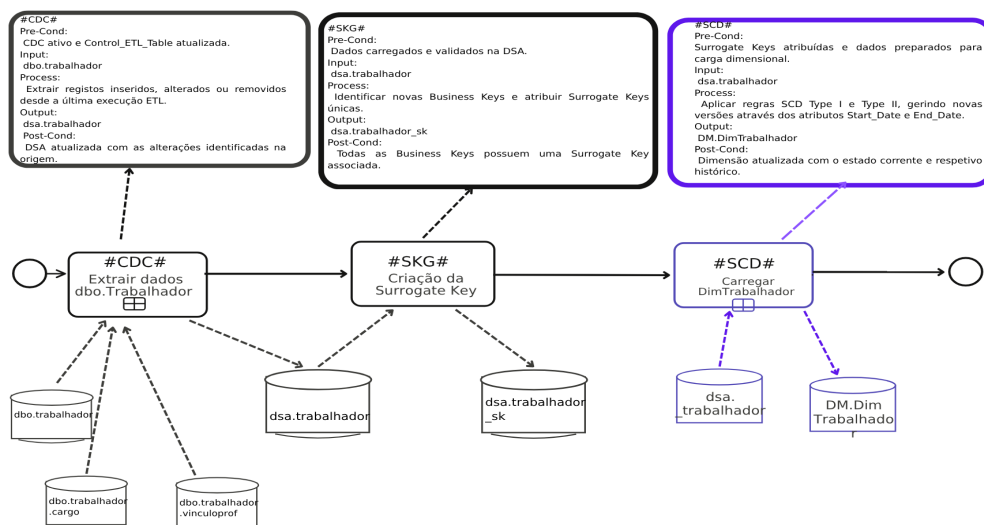


Figura 7 – Diagrama BPMN - Nível de Partner

5.5. Carregamento da Tabela de Factos

O carregamento da *FactMovimentoStock* baseia-se em duas tabelas de staging independentes, *dsa.Entrada* e *dsa.Saida*, que replicam a estrutura das respetivas tabelas operacionais e armazenam exclusivamente os registos da janela temporal correspondente à execução corrente.

Na carga inicial, são extraídos todos os registos disponíveis. Nas execuções subsequentes, são considerados apenas os registos com data posterior à última execução bem-sucedida, registada na tabela *dsa.Control_ETL_Table*. Como cada fonte é processada de forma independente, esta tabela inclui a coluna *TableName*, que permite manter um ponto de controlo distinto por tabela, evitando que a falha de um fluxo comprometa o outro e garantindo que cada execução retoma exatamente a partir do ponto de interrupção.

O *data flow* consolida as duas fontes através de um *UNION ALL*, seguido de uma conversão de tipos para assegurar a compatibilidade com o esquema de destino. São depois realizados *lookups* às tabelas de mapeamento de *Surrogate Keys*, de forma a resolver todas as Business Keys presentes na fonte.

O atributo *data_hora* é desdobrado em duas colunas derivadas, correspondentes à data e à hora, permitindo a associação às dimensões DimData e DimHora. Eventuais valores nulos nas chaves estrangeiras são substituídos pelas linhas especiais de membro desconhecido de cada dimensão.

Por fim, é calculada a métrica derivada *CustoTotal*, resultante da multiplicação do custo unitário pela quantidade movimentada, sendo os registos posteriormente carregados na tabela *DW.FactMovimentoStock*.